PAISAINSPECTOR

GenAI Powered Finance Assistance

PROJECT OVERVIEW

The GenAI-Based SMS Financial Intelligence System is an intelligent financial analysis platform that automatically extracts, processes, categorizes, and analyzes transactional SMS messages to generate meaningful financial insights using Artificial Intelligence.

In today's digital banking ecosystem, users receive transaction notifications through SMS for every debit, credit, purchase, rent payment, bill payment, or salary credit. However, this financial information remains unstructured and difficult to interpret manually over time. Users often struggle to understand:

- Where their money goes
- Which category consumes most income
- Monthly savings patterns
- Spending behaviour trends
- Budget imbalances

This project solves that problem by converting raw transactional SMS messages into structured financial intelligence.

SCENARIO 1: Automated Monthly Expense Analysis for a Working Individual

In today's digital banking environment, individuals receive multiple transaction-related SMS notifications daily, including salary credits, rent payments, grocery purchases, food deliveries, travel expenses, and subscription charges. Over a period of time, these messages accumulate and make it difficult for users to manually track where their money is being spent. A working professional earning ₹45,000 per month may not clearly understand how much is going towards food, rent, travel, or entertainment without manually checking each transaction message.

The GenAI Finance Assistant addresses this issue by automatically extracting financial information from SMS transaction data. The system processes raw SMS text, identifies important fields such as date, transaction type, merchant name, amount, and balance, and then categorizes the transactions into meaningful spending groups such as Food, Rent, Travel, Shopping, and Utilities. After organizing the data into structured format, the system performs financial analysis to calculate total income, total expenses, savings, and category-wise breakdown.

Finally, the AI model generates a clear and understandable summary of the user's financial behavior. Instead of manually analyzing hundreds of messages, the user receives a concise explanation of where most of the money is spent and how much savings remain. This transforms unstructured SMS data into actionable financial insights.

SCENARIO 2: AI-Based Personalized Financial Question Answering

Users often want answers to specific financial questions such as “Where does my money go mostly?”, “Am I spending too much on food?”, or “How much did I save this month?” Traditional budgeting apps may provide numerical reports, but they do not generate intelligent, conversational explanations based on user queries.

In this scenario, the GenAI Finance Assistant allows the user to ask natural language questions related to their personal transaction history. Once a question is submitted, the backend system retrieves processed transaction data from the CSV file generated through SMS parsing. It performs spending analysis to calculate totals and category distributions. This structured financial summary is then combined with the user’s question and sent to the Gemini AI model.

The AI generates a personalized response grounded in actual transaction data rather than generic advice. This ensures that the response reflects real spending behavior. The system therefore bridges the gap between raw financial data and intelligent conversational financial guidance.

SCENARIO 3: Financial Text Simplification for Better Understanding

Many individuals struggle to understand complex financial terminology found in bank messages, loan agreements, investment policies, credit card statements, or tax-related documents. Financial institutions often use technical language that may not be easily understandable for beginners or non-finance users.

The Explain Financial Text feature of the system allows users to paste complex financial text into the application. The text is then processed by the Gemini AI model using a prompt designed to simplify and explain financial terminology in clear and simple language. Instead of technical jargon, the AI produces beginner-friendly explanations that improve financial literacy.

For example, a statement containing interest rate terms or credit card charges can be transformed into a simple explanation describing what it means and how it affects the user financially. This feature enhances usability and ensures that the system is not only analytical but also educational.

ARCHITECTURE OVERVIEW

The GenAI Finance Assistant is built on a modular, service-oriented architecture that separates user interaction, financial data processing, and AI-driven reasoning into distinct layers. At the frontend, a lightweight interface developed using HTML, CSS, and JavaScript enables users to generate financial advice, explain financial text, and ask personalized financial questions. The interface communicates with a FastAPI backend through RESTful endpoints, ensuring smooth and scalable client–server interaction. Each user request is routed to a specific backend endpoint where inputs are validated before processing, maintaining consistency and reliability throughout the system.

At the core of the backend lies the financial intelligence layer, which transforms raw SMS transaction data into structured financial insights. Unstructured SMS messages are parsed using a custom extraction module, categorized through rule-based merchant mapping, and stored in a structured CSV format. Analytical modules compute total

income, total expenses, savings, and category-wise spending distributions. These structured financial summaries form the foundation for intelligent reasoning.

The AI reasoning layer is powered by Google's Gemini generative model. Carefully designed prompt templates combine real transaction analysis with user questions to generate personalized, context-aware financial responses. Instead of producing generic advice, the AI grounds its explanations in actual spending data processed by the system. The backend manages prompt construction, AI invocation, response validation, and error handling to ensure robustness and reliability.

Secure API key management through environment variables and structured configuration files ensures safe integration with external AI services. The modular design makes the architecture extensible, allowing future enhancements such as database integration, real-time SMS ingestion, dashboards, or advanced financial forecasting models. Overall, the architecture is scalable, maintainable, and suitable for deployment as a real-world intelligent financial assistant system.

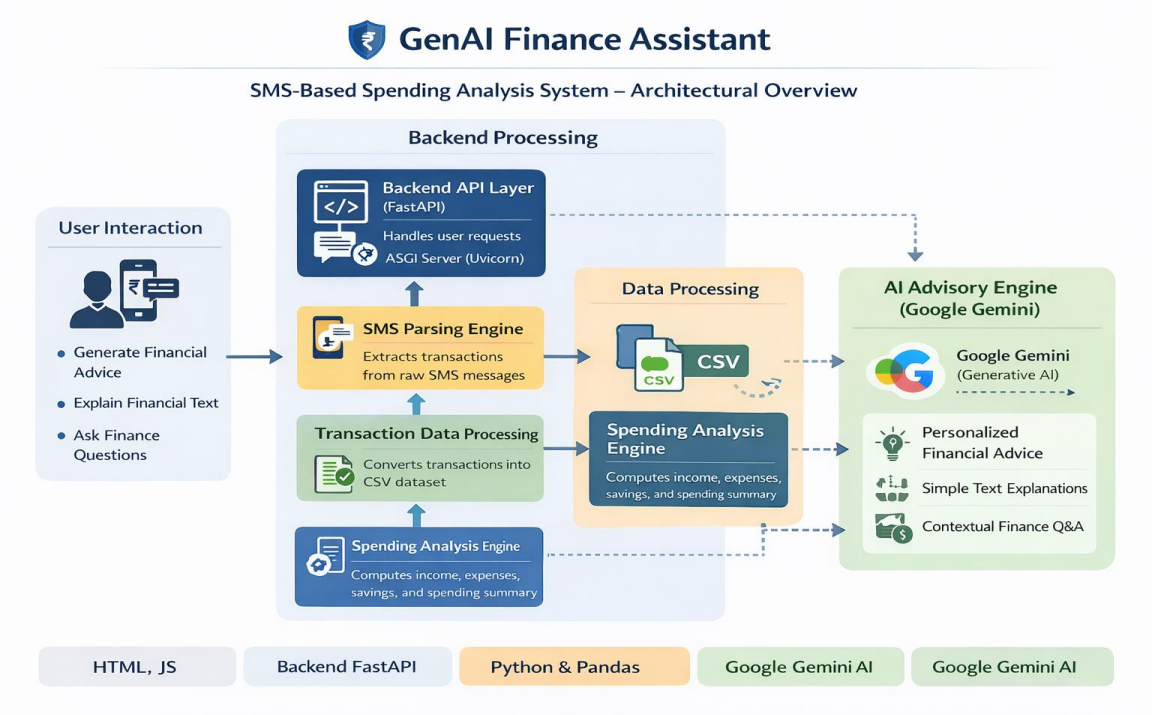


Fig - Architecture Diagram

CORE TECHNOLOGIES

- **HTML, CSS, JavaScript (Frontend Layer):** The frontend is built using standard web technologies to provide a clean, lightweight, and responsive user interface. HTML structures the application layout, CSS enhances visual presentation, and JavaScript handles dynamic interactions such as API calls, loading states, and real-time

response updates. The frontend communicates with the FastAPI backend via RESTful endpoints, ensuring smooth client server interaction without relying on heavy frameworks.

- **FastAPI (Backend API Framework):** FastAPI serves as the core backend framework responsible for handling HTTP requests, routing endpoints, validating user inputs, and orchestrating financial analysis and AI workflows. Its high performance, asynchronous capabilities, and automatic API documentation make it ideal for scalable AI integrated systems. It acts as the bridge between the frontend interface, transaction processing modules, and the Gemini AI model.
- **Google Gemini (Generative AI Model):** Gemini acts as the intelligent reasoning engine of the system. It generates financial advice, explains complex financial text, and answers personalized financial questions using structured prompts. Through prompt engineering, Gemini leverages real transaction summaries produced by the backend analysis layer to deliver contextual and data-driven responses instead of generic outputs.
- **Prompt Engineering Module:** A dedicated prompt template layer constructs structured and context-aware instructions for the Gemini model. This module ensures that real spending summaries are injected into AI prompts when answering user questions. It maintains clarity, consistency, and reliability in generated outputs by controlling response format and contextual grounding.
- **Custom SMS Parsing Engine:** A rule-based extraction module processes raw bank SMS messages and converts them into structured transaction records. It identifies transaction date, type (credited/debited), merchant name, amount, and available balance. This component transforms unstructured financial messages into machine-readable structured data.
- **Transaction Processing & Categorization Module:** After extraction, merchant names are mapped to predefined financial categories such as Food, Transport, Rent, Shopping, and Subscriptions. The processed data is stored in a structured CSV file for downstream analysis. This layer ensures that raw financial data becomes analytically usable.
- **Spending Analysis Engine (Python Logic Layer):** This module computes total income, total expenses, savings, category-wise spending, and transaction counts from structured CSV data. It forms the analytical backbone of the system and provides summarized financial insights that power AI-driven responses.
- **Pandas (Data Processing Library):** Used for reading and manipulating structured transaction data efficiently. It enables easy CSV handling, data aggregation, and preprocessing before feeding insights into the AI reasoning layer.
- **dotenv (Environment Management):** Environment variables are securely managed using dotenv to store sensitive credentials such as the Gemini API key. This prevents hardcoding of secrets and supports secure deployment practices.
- **Uvicorn (ASGI Server):** Uvicorn runs the FastAPI application as an ASGI server, enabling high-performance asynchronous request handling and development-time auto-reloading for rapid iteration.

COMPONENT-WISE ARCHITECTURE

Component	Description
User Interface (HTML, CSS, JavaScript)	Provides a lightweight web interface where users can generate financial advice, explain financial text, and ask finance-related questions. JavaScript manages API calls to the backend and dynamically updates results in real time.
Backend API Layer (FastAPI)	Serves as the central coordination layer that receives HTTP requests from the frontend, validates inputs using schemas, triggers analysis or AI workflows, and returns structured JSON responses.
SMS Parsing Engine	Extracts structured transaction details (date, type, merchant, amount, balance) from raw

	bank SMS messages. Converts unstructured text into structured transaction records.
Transaction Processing Module	Converts parsed SMS transactions into a structured CSV dataset and maps merchants into spending categories such as Food, Transport, Rent, Shopping, and Subscriptions.
Spending Analysis Engine	Computes total income, total expenses, savings, transaction count, and category-wise spending from the processed transaction dataset.
AI Advisory Engine (Google Gemini)	Uses the Gemini generative AI model to generate general financial advice, explain financial text, and answer personalized financial questions using structured spending summaries.
Prompt Engineering Module	Builds structured prompts for general advice, text explanation, and data-driven financial Q&A. Injects transaction summaries into prompts for contextual AI reasoning.
Data Processing Layer (Pandas)	Reads and processes structured CSV transaction data for financial analysis and summary generation.
Environment & Configuration Manager (dotenv)	Securely loads environment variables such as the Gemini API key and manages application-level configuration.
ASGI Server (Uvicorn)	Runs the FastAPI backend server and handles asynchronous request processing during development and deployment.
Error Handling & Safety Layer	Captures API failures, data processing issues, and connectivity errors, returning structured error messages without crashing the system.

PRE-REQUISITES

The successful development, execution, and deployment of the GenAI Finance Assistant (SMS-Based Spending Analysis System) require the following hardware, software, and technical prerequisites. These ensure stable AI processing, backend API execution, SMS parsing, transaction analysis, and frontend interaction.

1. Python Environment Setup

Python is the core programming language used for backend development, AI integration, SMS parsing, and financial data analysis.

- **Required Version:** Python 3.10 or above (Recommended: 3.11)
- **Purpose:**
  - Backend API development (FastAPI)
  - SMS transaction parsing
  - Spending analysis computation
  - Google Gemini AI integration
    - **Recommendation:** Use a dedicated virtual environment (venv) to avoid dependency conflicts.

Verification Command:

python --version

2. Virtual Environment (venv)

A virtual environment ensures isolated dependency management and project stability.

• Prevents conflicts with system-wide Python packages
• Improves deployment reliability
• Makes project sharing easier

Commands:

python -m venv .venv

Activate:

Windows:

.venv\Scripts\activate

Linux / macOS:

source .venv/bin/activate

3. Required Python Libraries

All required dependencies must be installed before running the project.

Key Libraries Used:

Library	Purpose
FastAPI	Backend REST API framework
Uvicorn	ASGI server for FastAPI
Google-generativeai / google-genai	Gemini AI integration
Pandas	CSV processing & financial data analysis
Python-dotenv	Secure environment variable management
Pydantic	Data validation & request schemas
Requests (Optional)	API testing
CSV (Built-in)	SMS-to-transaction conversion

Installation Command:

pip install -r requirements.txt

Or manually:

pip install fastapi uvicorn pandas python-dotenv google-generativeai pydantic

4. Google Gemini API Setup

Your system depends on Google Gemini for AI-powered financial advice and question answering.

Steps:

a. Generate API key from Google AI Studio.
b. Create a .env file in project root.
c. Add:

GEMINI_API_KEY=your_api_key_here

Purpose:

- Generates financial advice
- Explains financial text
- Answers data-driven financial questions
- Uses real transaction summaries

5. FastAPI & Uvicorn Setup (Backend)

FastAPI handles all API endpoints and coordinates:

- SMS processing
- Spending analysis
- AI prompt generation
- Response formatting

Run Backend:

python -m uvicorn backend.main:app --reload

Backend runs at:

http://127.0.0.1:8000

6. Frontend Setup (HTML, CSS, JavaScript)

Your frontend is built using:

- HTML - Structure
- CSS - Styling
- JavaScript - API communication

Purpose:

- Send requests to backend
- Display AI responses
- Provide financial interaction UI

Run by simply opening:

frontend/index.html

Or use VS Code Live Server for better testing.

7. SMS Dataset Availability

The system requires a raw SMS transaction dataset.

- File Location:

backend/data/raw/sms_data.txt

- Requirements:
    - Must contain bank transaction messages
    - Must follow consistent structure (Rs., debited, credited, etc.)

8. Processed Transaction CSV

Generated automatically after SMS parsing.

- File Location:

backend/data/processed/transactions.csv

- Contains:
    - Date
    - Type (Credited/Debited)
    - Merchant
    - Category
    - Amount
    - Balance

9. Git & GitHub (Version Control)

Required for:

- Source code management
- Collaboration
- Project submission

Install:

- Git: https://git-scm.com/
- GitHub account for repository hosting

10. Recommended Development Tools

Tool	Purpose
Visual Studio Code	Development & debugging
Postman	API testing
Chrome / Edge	Frontend testing
Terminal / PowerShell	Backend execution

PROJECT FLOW

1. Environment Setup and Dependency Configuration

- **Activity 1.1:** Install Python 3.10+ and create a dedicated virtual environment (venv) to ensure clean and isolated dependency management.
- **Activity 1.2:** Install required libraries including FastAPI, Uvicorn, Pandas, Google Gemini SDK, python-dotenv, and Pydantic using the requirements.txt file.
- **Activity 1.3:** Configure environment variables by creating a .env file and securely storing the GEMINI_API_KEY.
- **Activity 1.4:** Verify successful backend setup by running the FastAPI server using Uvicorn and testing endpoints via browser or Postman.
- **Activity 1.5:** Organize project structure into modular folders including:
    - backend/analysis
    - backend/extraction
    - backend/categorization
    - backend/genai
    - backend/data/raw
    - backend/data/processed
    - frontend/
- **Activity 1.6:** Configure Git and GitHub for version control and academic submission readiness.

2. Synthetic SMS Dataset Creation

- **Activity 2.1:** Create a structured synthetic dataset simulating real-world bank SMS transaction messages including:
    - Salary credits
    - Rent payments
    - Food orders (Swiggy, BigBasket)
    - Shopping (Amazon)
    - Transport (Uber)
    - Subscriptions (Netflix, Spotify)
    - Utilities (Airtel)
- **Activity 2.2:** Ensure SMS messages follow consistent banking format:
    - Transaction type (Credited/Debited)
    - Amount (Rs.)
    - Merchant name
    - Date
    - Available balance

- **Activity 2.3:** Store the dataset inside:

backend/data/raw/sms_data.txt

- **Activity 2.4:** Validate dataset realism to ensure spending patterns resemble real monthly financial behavior.

3. SMS Parsing and Transaction Extraction

- **Activity 3.1:** Develop SMS parsing logic to extract structured transaction details from raw SMS text.
- **Activity 3.2:** Extract:
    - Date
    - Transaction Type
    - Merchant
    - Amount
    - Balance
- **Activity 3.3:** Implement category mapping logic to classify merchants into categories:
    - Food
    - Rent
    - Shopping
    - Transport
    - Entertainment
    - Utilities
- **Activity 3.4:** Convert parsed transactions into structured CSV format.
- **Activity 3.5:** Store processed file in:

backend/data/processed/transactions.csv

4. Spending Analysis Engine Development

- **Activity 4.1:** Develop financial analysis logic to compute:
    - Total Income
    - Total Expenses
    - Savings
    - Category-wise spending
    - Total transaction count
- **Activity 4.2:** Identify top spending category dynamically.
- **Activity 4.3:** Calculate financial health indicators (income vs expense difference).

- **Activity 4.4:** Ensure error-safe processing for empty or malformed datasets.

5. AI Integration using Google Gemini

- **Activity 5.1:** Integrate Google Gemini API for:
    - General financial advice generation
    - Explaining financial text
    - Answering user financial questions
- **Activity 5.2:** Design structured prompt templates for:
    - Beginner-friendly explanations
    - Data-driven financial reasoning
    - Personalized spending advice
- **Activity 5.3:** Combine spending analysis results with AI prompts to generate intelligent responses based on real transaction data.
- **Activity 5.4:** Implement safe response extraction and error handling for API failures.
- **Activity 5.5:** Ensure AI responses use correct currency format (₹ instead of $).

6. Backend API Development (FastAPI)

- **Activity 6.1:** Create RESTful endpoints:
    - /explain/general
    - /explain/text
    - /ask
- **Activity 6.2:** Use Pydantic schemas for input validation.
- **Activity 6.3:** Connect endpoints with:
    - Spending Analysis Engine
    - Gemini AI Module
- **Activity 6.4:** Enable CORS middleware for frontend-backend communication.
- **Activity 6.5:** Implement structured JSON responses for clean frontend integration.

7. Frontend Development (HTML, CSS, JavaScript)

- **Activity 7.1:** Develop a clean web interface with:
    - General advice section
    - Financial text explanation section
    - Ask question section
- **Activity 7.2:** Use JavaScript fetch() API to communicate with backend endpoints.
- **Activity 7.3:** Implement loading indicators and error messages.
- **Activity 7.4:** Display AI responses in readable format.
- **Activity 7.5:** Test full interaction flow between frontend and backend.

8. Testing, Debugging, and Optimization

- **Activity 8.1:** Perform unit testing of:
    - SMS parsing
    - CSV generation
    - Spending analysis calculations
- **Activity 8.2:** Validate frontend-backend communication.
- **Activity 8.3:** Debug issues such as:
    - API key errors
    - Currency mismatch
    - DataFrame vs file path errors
    - Missing library errors (pandas)
- **Activity 8.4:** Optimize prompt efficiency to reduce token usage.
- **Activity 8.5:** Conduct end-to-end testing for:
    - Salary vs expense correctness
    - Category accuracy
    - AI reasoning based on real data

9. Final System Integration and Validation

- Ensure:
    - SMS → CSV conversion works correctly
    - CSV → Spending summary works correctly

- Spending summary → Gemini prompt works correctly
- Backend → Frontend response rendering works correctly

- Validate system in real demo scenario:

  - User uploads SMS data
  - System processes transactions
  - AI provides intelligent financial insights

MILESTONE 1 - Gemini API Infrastructure Initialization & Backend Foundation

This foundational milestone establishes the complete technical environment required to build and run the GenAI Finance Assistant. The objective of this stage was to ensure that all dependencies, backend frameworks, SMS processing modules, and Google Gemini AI integrations were correctly configured before implementing financial intelligence features.

This milestone was critical because without proper API configuration, environment setup, and modular backend structuring, the system would not be able to:

- Parse SMS transaction data
- Generate spending analysis
- Communicate with Google Gemini
- Serve responses to the frontend

By the end of this stage, the backend server was running successfully and the Gemini model was generating test responses.

- **Activity 1.1: Obtain Google Gemini API Key**

To enable AI-powered financial reasoning, Google Gemini API access was required.

Step 1: Visit Google AI Studio

Open:

https://aistudio.google.com

Step 2: Login with Google Account

Sign in using a valid Google account.

Step 3: Navigate to API Keys

- Click on profile/dashboard

- Select **"Get API Key"**

Step 4: Create API Key

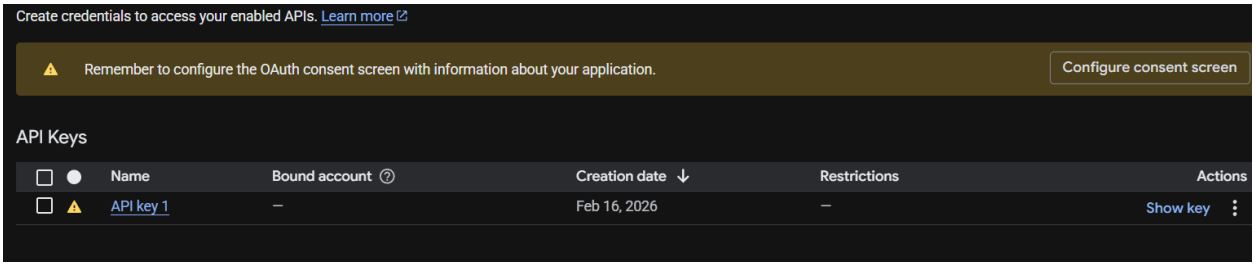
- Click **Create API Key**
- Name it:

  GenAI_Finance_Assistant_Project

Step 5: Store API Key Securely

Create a .env file inside the backend root directory:

GEMINI_API_KEY=your_generated_api_key_here



- **Activity 1.2: Environment Setup & Dependency Installation**

To ensure clean dependency management, a virtual environment was created.

Step 1: Create Virtual Environment

python -m venv venv

Step 2: Activate Environment (Windows)

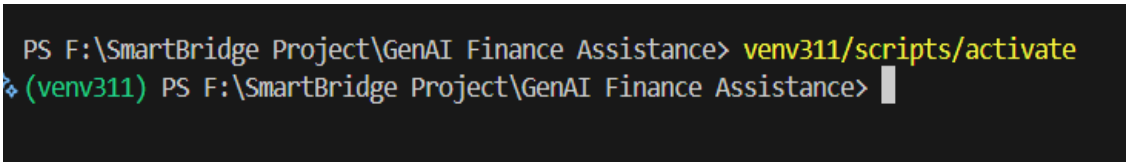
venv\Scripts\activate

Step 3: Install Required Libraries

pip install fastapi uvicorn pandas python-dotenv google-genai pydantic

Step 4: Verify Installation

pip list



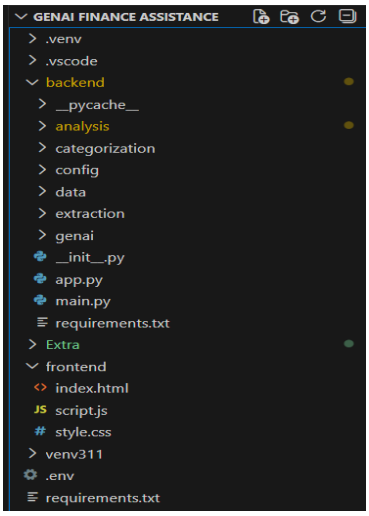
- **Activity 1.3: Backend Project Structure Initialization**

  A modular project structure was designed to maintain scalability and clarity.

```
backend/
├── analysis/
│   ├── spending_analysis.py
│   └── transaction_processor.py
├── extraction/
│   └── sms_parser.py
├── categorization/
│   └── category_mapper.py
├── genai/
│   ├── prompt_templates.py
│   └── explanation_engine.py
├── config/
│   └── settings.py
├── data/
│   ├── raw/
│   └── processed/
└── main.py
```

  This modular separation ensures:

  - Clean responsibility separation
  - Easy debugging
  - Future feature expansion
  - Better academic presentation



- **Activity 1.4: Gemini Connectivity Validation**

  After setup, Gemini connectivity was tested using a simple test script.

  Test Code:

  from google import genai

  client = genai.Client(api_key="YOUR_API_KEY")

  response = client.models.generate_content(
  model="gemini-2.5-flash",
  contents="Say hello in one sentence."
)

  print(response.text)

  Expected Output:

  Hello! I hope you're having a great day.

  This confirmed:

  - API key is valid
  - Model access is enabled
  - SDK integration is correct

```
PS F:\SmartBridge Project\GenAI Finance Assistance> venv311/scripts/activate
(venv311) PS F:\SmartBridge Project\GenAI Finance Assistance> python
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> from google import genai
>>>
>>> client = genai.Client(api_key="AIzaSyCZxGpno1Gb9yM2h86r_rIWnSfVkakPOMI")
>>>
>>> response = client.models.generate_content(
...     model="gemini-2.5-flash",
...     contents="Say hello in one sentence."
... )
>>>
>>> print(response.text)
Hello there!
>>>
```

- **Activity 1.5: FastAPI Backend Initialization**

  The FastAPI server was initialized using:

  python -m uvicorn backend.main:app --reload

  Server runs at:

  http://127.0.0.1:8000

  Tested endpoint:

  /explain/general

  Browser Test:

  http://127.0.0.1:8000/explain/general

  This verified:

  - Backend server running
  - API endpoints functional
  - AI integration operational

```
(.venv) PS F:\SmartBridge Project\GenAI Finance Assistance> python -m uvicorn backend.main:app --reload
INFO:     Application shutdown complete.
INFO:     Finished server process [2936]
INFO:     Stopping reloader process [22192]
(.venv) PS F:\SmartBridge Project\GenAI Finance Assistance> python -m uvicorn backend.main:app --reload
INFO:     Will watch for changes in these directories: ['F:\\SmartBridge Project\\GenAI Finance Assistan
ce']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [20804] using StatReload
✅ GEMINI API KEY Loaded Successfully
```

MILESTONE 2 - SMS Transaction Extraction & Financial Data Processing Engine

This milestone focuses on building the core financial data pipeline of the GenAI Finance Assistant. The objective of this stage was to transform unstructured raw SMS transaction messages into structured financial data that can be analysed programmatically.

Unlike generic AI projects, this system works on realistic banking SMS transaction messages, which are messy, inconsistent, and unstructured. Therefore, before applying AI reasoning, we first built a deterministic financial processing engine.

By the end of this milestone, the system was capable of:

- Reading raw SMS transaction data
- Extracting structured transaction fields
- Categorizing merchants
- Saving transactions to CSV
- Performing spending analysis
- Generating financial summaries

- **Activity 2.1: Synthetic SMS Dataset Creation**

  Since accessing real bank SMS data is not feasible for privacy reasons, we created a synthetic but realistic SMS dataset that mimics actual Indian banking transaction messages.

  Example SMS:

  Rs.45000 credited to your A/c XX2345 on 01-09-2025. Salary credited
  Rs.250 debited from A/c XX2345 on 04-09-2025 at SWIGGY. Avl bal Rs.44750
  Rs.120 debited from A/c XX2345 on 04-09-2025 at UBER. Avl bal Rs.44630

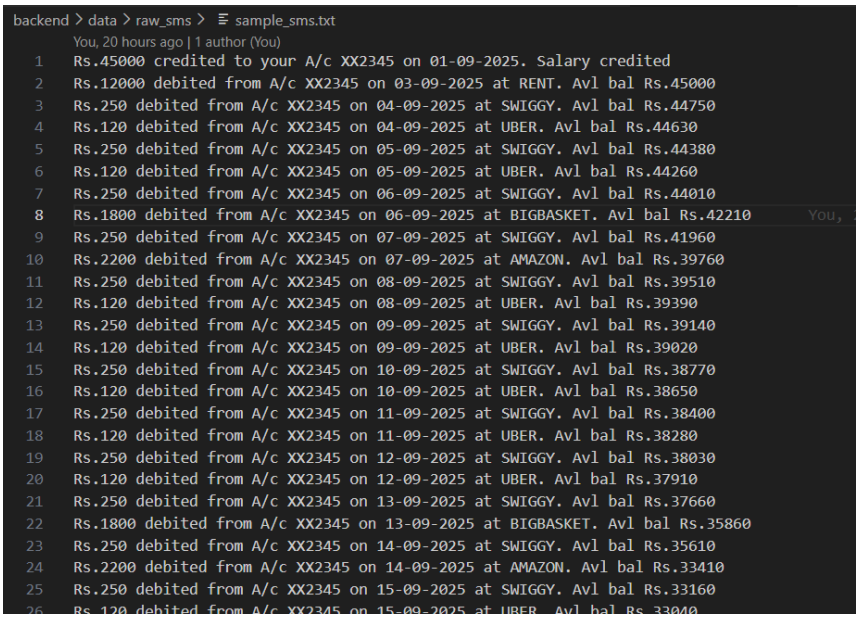
  The dataset contains:

  - Salary credits
  - Rent payments
  - Swiggy food orders
  - Uber rides
  - Amazon shopping
  - Netflix & Spotify subscriptions
  - Airtel bills

  This dataset simulates real-life monthly spending behaviour across multiple months.

backend > data > raw_sms > sample_sms.txt

You, 20 hours ago | 1 author (You)



```
Rs.45000 credited to your A/c XX2345 on 01-09-2025. Salary credited
Rs.12000 debited from A/c XX2345 on 03-09-2025 at RENT. Avl bal Rs.45000
Rs.250 debited from A/c XX2345 on 04-09-2025 at SWIGGY. Avl bal Rs.44750
Rs.120 debited from A/c XX2345 on 04-09-2025 at UBER. Avl bal Rs.44630
Rs.250 debited from A/c XX2345 on 05-09-2025 at SWIGGY. Avl bal Rs.44380
Rs.120 debited from A/c XX2345 on 05-09-2025 at UBER. Avl bal Rs.44260
Rs.250 debited from A/c XX2345 on 06-09-2025 at SWIGGY. Avl bal Rs.44010
Rs.1800 debited from A/c XX2345 on 06-09-2025 at BIGBASKET. Avl bal Rs.42210
Rs.250 debited from A/c XX2345 on 07-09-2025 at SWIGGY. Avl bal Rs.41960
Rs.2200 debited from A/c XX2345 on 07-09-2025 at AMAZON. Avl bal Rs.39760
Rs.250 debited from A/c XX2345 on 08-09-2025 at SWIGGY. Avl bal Rs.39510
Rs.120 debited from A/c XX2345 on 08-09-2025 at UBER. Avl bal Rs.39390
Rs.250 debited from A/c XX2345 on 09-09-2025 at SWIGGY. Avl bal Rs.39140
Rs.120 debited from A/c XX2345 on 09-09-2025 at UBER. Avl bal Rs.39020
Rs.250 debited from A/c XX2345 on 10-09-2025 at SWIGGY. Avl bal Rs.38770
Rs.120 debited from A/c XX2345 on 10-09-2025 at UBER. Avl bal Rs.38650
Rs.250 debited from A/c XX2345 on 11-09-2025 at SWIGGY. Avl bal Rs.38400
Rs.120 debited from A/c XX2345 on 11-09-2025 at UBER. Avl bal Rs.38280
Rs.250 debited from A/c XX2345 on 12-09-2025 at SWIGGY. Avl bal Rs.38030
Rs.120 debited from A/c XX2345 on 12-09-2025 at UBER. Avl bal Rs.37910
Rs.250 debited from A/c XX2345 on 13-09-2025 at SWIGGY. Avl bal Rs.37660
Rs.1800 debited from A/c XX2345 on 13-09-2025 at BIGBASKET. Avl bal Rs.35860
Rs.250 debited from A/c XX2345 on 14-09-2025 at SWIGGY. Avl bal Rs.35610
Rs.2200 debited from A/c XX2345 on 14-09-2025 at AMAZON. Avl bal Rs.33410
Rs.250 debited from A/c XX2345 on 15-09-2025 at SWIGGY. Avl bal Rs.33160
Rs.120 debited from A/c XX2345 on 15-09-2025 at UBER. Avl bal Rs.33040
```

- **Activity 2.2: SMS Parsing Engine Development**

  We built a custom parser to extract structured data from raw SMS messages.

  File:

  backend/extraction/sms_parser.py

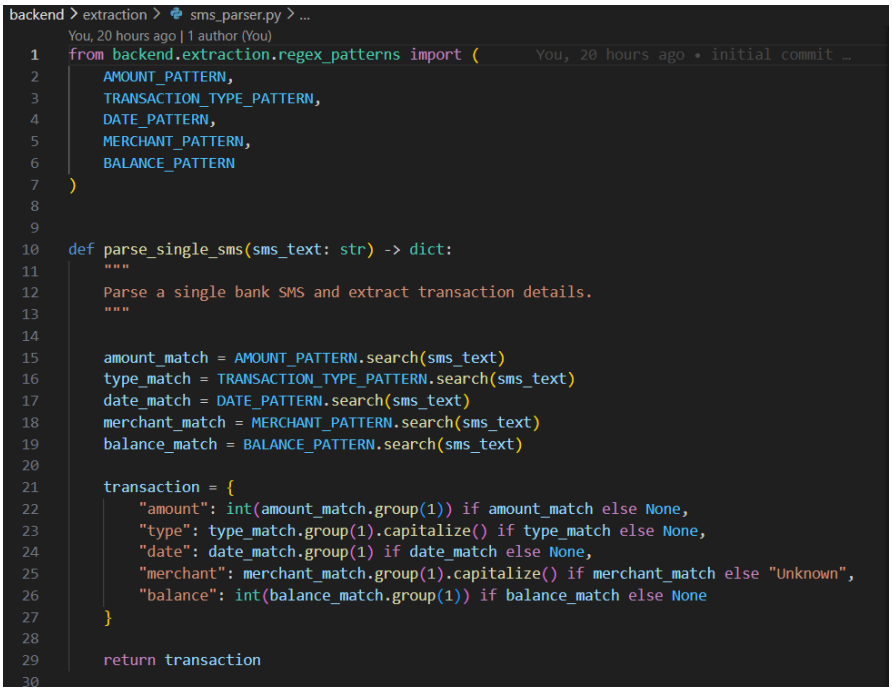
  The parser extracts:

  - Date
  - Transaction type (Credited/Debited)
  - Merchant name
  - Amount
  - Balance

  Example extracted structure:

  ```
  {
    "date": "04-09-2025",
    "type": "Debited",
    "merchant": "SWIGGY",
    "amount": 250,
    "balance": 44750
  }
  ```

  This step converts messy SMS text into clean structured Python dictionaries.




```python
def parse_sms_file(file_path: str) -> list:
    """
    Parse an SMS text file and return list of transactions.
    """

    transactions = []

    with open(file_path, "r") as file:
        for line in file:
            sms = line.strip()
            if sms:
                transactions.append(parse_single_sms(sms))

    return transactions
```

- **Activity 2.3: Merchant Category Mapping**

  After extracting merchants, we categorized them logically.

  File:

  backend/categorization/category_mapper.py

  Example mapping:

  SWIGGY → Food

UBER → Transport
AMAZON → Shopping
BIGBASKET → Groceries
NETFLIX → Entertainment
AIRTEL → Utilities
RENT → Housing

This classification allows us to compute:

- Category-wise expenses
- Top spending category
- Financial behaviour patterns

```
You, 20 hours ago | 1 author (You)
def get_category(merchant: str) -> str:
    """
    Map merchant name to expense category.
    """

    merchant = merchant.lower()

    if merchant in ["swiggy", "zomato", "bigbasket"]:
        return "Food"
    elif merchant in ["uber", "ola", "irctc"]:
        return "Transport"
    elif merchant in ["amazon", "flipkart", "myntra", "reliance"]:
        return "Shopping"
    elif merchant in ["netflix", "spotify", "amazonprime"]:
        return "Entertainment"
    elif merchant in ["airtel", "jio", "bses", "tatapower"]:
        return "Utilities"
    elif merchant in ["salary", "cashback", "refund"]:
        return "Income"
    else:
        return "Others"
```

- **Activity 2.4: CSV Transaction Storage Engine**

Next, we built a transaction processor that converts parsed SMS data into structured CSV format.

File:

backend/analysis/transaction_processor.py

The function:

process_sms_to_csv(sms_file_path, output_csv_path)

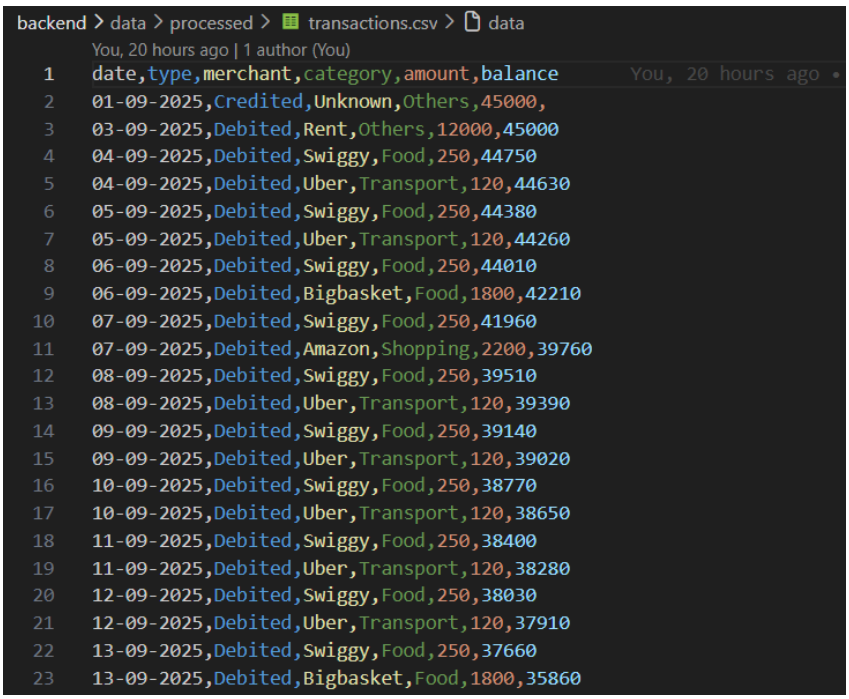
Creates CSV structure:

date	type	merchant	category	amount	balance
01-09-2025	Credited	Salary	Income	45000	45000
04-09-2025	Debited	SWIGGY	Food	250	44750

This CSV file is stored in:

backend/data/processed/transactions.csv

This structured dataset becomes the base for spending analysis.



- **Activity 2.5: Spending Analysis Engine Implementation**

  File:

  backend/analysis/spending_analysis.py

  We implemented logic to compute:

  - Total Income
  - Total Expense
  - Savings
  - Category-wise spending
  - Transaction count

  Core logic:

```
if tx_type == "Credited":
    total_income += amount
elif tx_type == "Debited":
    total_expense += amount
    category_expense[category] += amount
```

Final summary structure:

```
{
    "total_income": 180000,
    "total_expense": 129348,
    "savings": 50652,
    "category_expense": {
        "Food": 51000,
        "Rent": 48000,
        "Transport": 7000
    }
}
```

This structured summary is later passed to Gemini for intelligent explanation.

backend > analysis > spending_analysis.py > ...

```python
def analyze_spending(csv_file_path: str, month: str = None) -> dict:
    total_income = 0
    total_expense = 0
    category_expense = defaultdict(int)
    transaction_count = 0

    with open(csv_file_path, "r") as csvfile:
        reader = csv.DictReader(csvfile)

        for row in reader:
            date = row["date"]  # format: 01-09-2025

            if month:
                if f"-{month}-" not in date:
                    continue

            amount = int(row["amount"])
            tx_type = row["type"]
            category = row["category"]

            transaction_count += 1

            if tx_type == "Credited":
                total_income += amount
            elif tx_type == "Debited":
                total_expense += amount
                category_expense[category] += amount

    savings = total_income - total_expense

    return {
        "total_income": total_income,
        "total_expense": total_expense,
        "savings": savings,
        "category_expense": dict(category_expense),
        "transaction_count": transaction_count
    }
```

- **Activity 2.6: Debugging & Error Handling**

  During development, we encountered:

  - DataFrame vs CSV path mismatch
  - API Key environment conflict
  - ModuleNotFoundError (pandas)
  - Incorrect analysis key names

  We resolved them by:

  - Passing CSV file path instead of DataFrame
  - Installing missing dependencies
  - Correcting dictionary keys
  - Adding try–except blocks

  This improved system stability significantly.

MILESTONE 3: AI Financial Intelligence Integration using Google Gemini

This milestone represents the transformation of the system from a traditional financial data processor into an AI-powered intelligent financial assistant.

In Milestone 2, we built the structured financial data engine.
In Milestone 3, we integrated Google Gemini (Generative AI) to:

- Interpret financial summaries
- Answer user questions based on real spending data
- Explain financial text in simple language
- Provide personalized financial insights

By the end of this milestone, the system was capable of combining:

Deterministic Financial Analysis + Generative AI Intelligence

- Activity 3.1: Gemini API Infrastructure Integration

  We integrated Google Gemini using the official Python SDK.

  File:

  backend/genai/explanation_engine.py

  AI Client Initialization:

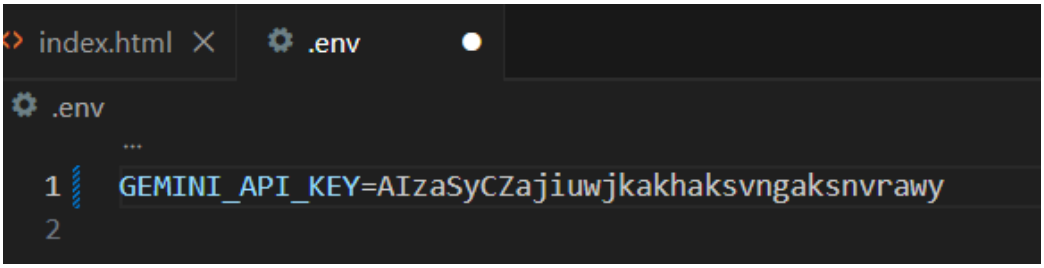
from google import genai
from backend.config.settings import GEMINI_API_KEY

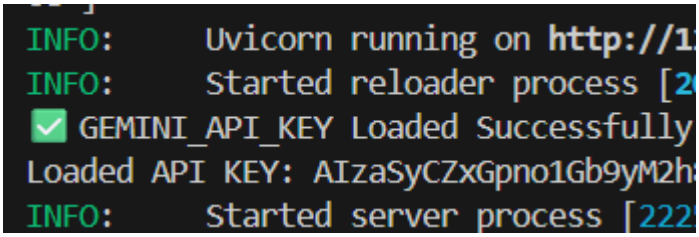
client = genai.Client(api_key=GEMINI_API_KEY)

We securely loaded the API key from:

.env

And validated it during startup.





- **Activity 3.2: Prompt Engineering Design**

  Instead of sending raw user input to the AI, we designed structured prompt templates.

  File:

  backend/genai/prompt_templates.py

  We created 3 prompt types:

  1. **General Financial Advice Prompt**

  GENERAL_EXPLANATION_PROMPT

  Purpose:

  - Provide beginner-friendly financial guidance.

  2. **Financial Text Explanation Prompt**

  TEXT_EXPLANATION_TEMPLATE

  Purpose:

  - Convert complex financial jargon into simple language.

  Example input:

  Compound interest allows your investment to grow exponentially...

  AI Output:

  - Simple explanation in easy words.

  3. **Question Answering Prompt**

  QUESTION_TEMPLATE

  Purpose:

  - Answer general finance questions clearly and simply.

```
You, 21 hours ago | 1 author (You)
from google import genai
from backend.config.settings import GEMINI_API_KEY
from backend.genai.prompt_templates import (
    GENERAL_EXPLANATION_PROMPT,
    TEXT_EXPLANATION_TEMPLATE,
)
from backend.analysis.spending_analysis import analyze_spending
import re

print("Loaded API KEY:", GEMINI_API_KEY)

client = genai.Client(api_key=GEMINI_API_KEY)


def _generate(prompt: str):
    try:
        print("Sending request to Gemini...")

        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt,
        )

        if hasattr(response, "text") and response.text:
            return response.text

        if hasattr(response, "candidates") and response.candidates:
            candidate = response.candidates[0]
            if candidate.content and candidate.content.parts:
```

```
def _generate(prompt: str):
            candidate = response.candidates[0]
            if candidate.content and candidate.content.parts:
                return candidate.content.parts[0].text

        return "Gemini returned empty response."

    except Exception as e:
        print("ERROR:", str(e))
        return f"Gemini API Error: {str(e)}"


# ==============================
# General Advice
# ==============================
def generate_general_explanation():
    return _generate(GENERAL_EXPLANATION_PROMPT)


# ==============================
# Explain Text
# ==============================
def explain_text(text: str):
    return _generate(TEXT_EXPLANATION_TEMPLATE.format(text=text))


MONTH_MAP = {
    "january": "01",
    "february": "02",
```

```
    "february": "02",
    "march": "03",
    "april": "04",
    "may": "05",
    "june": "06",
    "july": "07",
    "august": "08",
    "september": "09",
    "october": "10",
    "november": "11",
    "december": "12",
}

def detect_month(question: str):
    question = question.lower()
    for month_name, month_number in MONTH_MAP.items():
        if month_name in question:
            return month_number
    return None

# ==============================
# Ask Question Using REAL SMS DATA
# ==============================
def answer_question_with_data(question: str):
    try:
        csv_path = "backend/data/processed/transactions.csv"

        month = detect_month(question)

        analysis_result = analyze_spending(csv_path, month)
```

```
def answer_question_with_data(question: str):
        csv_path = "backend/data/processed/transactions.csv"

        month = detect_month(question)

        analysis_result = analyze_spending(csv_path, month)

        summary_text = f"""
All amounts are in Indian Rupees (₹).

Total Income: ₹{analysis_result['total_income']}
Total Expense: ₹{analysis_result['total_expense']}
Savings: ₹{analysis_result['savings']}
Category Expenses: {analysis_result['category_expense']}
"""

        full_prompt = f"""
You are a financial assistant.
Use the real transaction data below to answer the question.

{summary_text}

Question: {question}
"""

        return _generate(full_prompt)

    except Exception as e:
        return f"Data Processing Error: {str(e)}"
```

- **Activity 3.3: AI Response Generation Engine**

  We built a centralized function:

def _generate(prompt: str):

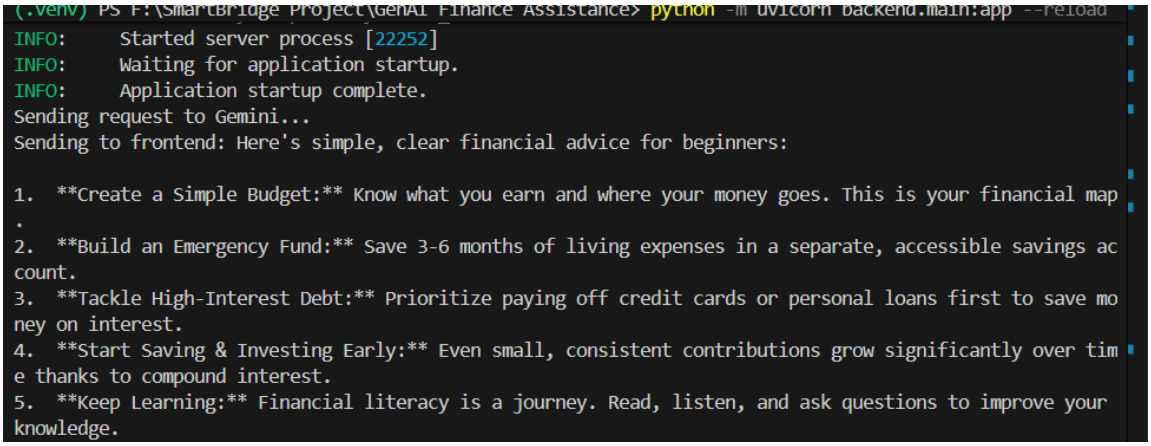
This function:

1. Sends request to Gemini
2. Extracts text safely
3. Handles missing responses
4. Catches API errors

Safe extraction logic:

if hasattr(response, "text") and response.text:
*   return response.text*

This prevents crashes due to unexpected AI output structure.



- **Activity 3.4: Real Financial Data + AI Combination**

This is where the project becomes unique.

Instead of answering blindly, we:

1. Loaded real transaction CSV
2. Ran spending analysis
3. Generated financial summary
4. Injected summary into Gemini prompt

Example:

df = pd.read_csv("backend/data/processed/transactions.csv")
analysis_result = analyze_spending("backend/data/processed/transactions.csv")

Then constructed structured summary:

summary_text = f"""
Total Income: {analysis_result['total_income']}
Total Expense: {analysis_result['total_expense']}
Savings: {analysis_result['savings']}
Top Category: {analysis_result['top_category']}
"""

Then passed to Gemini:

Use the following real spending data to answer the question:
...
Question: {question}

Now AI answers are:

- Context-aware
- Personalized
- Based on user's actual financial behaviour

	A	B	C	D	E	F	G
1	date	type	merchant	category	amount	balance	
2	########	Credited	Unknown	Others	45000		
3	########	Debited	Rent	Others	12000	45000	
4	########	Debited	Swiggy	Food	250	44750	
5	########	Debited	Uber	Transport	120	44630	
6	########	Debited	Swiggy	Food	250	44380	
7	########	Debited	Uber	Transport	120	44260	
8	########	Debited	Swiggy	Food	250	44010	
9	########	Debited	Bigbasket	Food	1800	42210	
10	########	Debited	Swiggy	Food	250	41960	
11	########	Debited	Amazon	Shopping	2200	39760	
12	########	Debited	Swiggy	Food	250	39510	
13	########	Debited	Uber	Transport	120	39390	
14	########	Debited	Swiggy	Food	250	39140	
15	########	Debited	Uber	Transport	120	39020	
16	########	Debited	Swiggy	Food	250	38770	
17	########	Debited	Uber	Transport	120	38650	
18	########	Debited	Swiggy	Food	250	38400	
19	########	Debited	Uber	Transport	120	38280	
20	########	Debited	Swiggy	Food	250	38030	
21	########	Debited	Uber	Transport	120	37910	
22	########	Debited	Swiggy	Food	250	37660	

- **Activity 3.5: Error Handling & Debugging Phase**

During integration, we encountered:

- API Key Invalid Errors

Solved by:

- Correct .env loading
- Removing old API keys

- 404 Model Not Found

Solved by:

- Using gemini-2.5-flash

- DataFrame vs CSV Path Error

Error:

expected str, bytes or os.PathLike object, not DataFrame

Solved by:
Passing CSV path instead of DataFrame.

- Incorrect Spending Calculation

Food showing ₹51000

Root Cause:

- Multi-month accumulation
- Repeated Swiggy entries
- Category mapping logic

Fixed by:

- Correct spending analysis logic
- Validating category sums

- **Activity 3.6: Financial Text Explanation Feature**

  This feature allows user to paste:

  - Bank statement explanation
  - Investment article
  - Loan policy text
  - Insurance terms

And system converts it into:

- Beginner-friendly explanation
- Short summary
- Clear understanding

This increases project versatility beyond transaction analysis.

```
merchant = merchant.lower()

if merchant in ["swiggy", "zomato", "bigbasket"]:
    return "Food"
elif merchant in ["uber", "ola", "irctc"]:
    return "Transport"
elif merchant in ["amazon", "flipkart", "myntra", "reliance"]:
    return "Shopping"
elif merchant in ["netflix", "spotify", "amazonprime"]:
    return "Entertainment"
elif merchant in ["airtel", "jio", "bses", "tatapower"]:
    return "Utilities"
elif merchant in ["salary", "cashback", "refund"]:
    return "Income"
else:
    return "Others"
```

- **Activity 3.7: Question Answering Based on Spending Data**

This is the most powerful feature.

Example Questions:

- Where am I spending most of my money?
- How can I reduce my food expenses?
- Am I saving enough?
- How can I increase my savings?

AI now answers using real data instead of generic advice.

MILESTONE 4: Backend API Orchestration & Frontend Integration

This milestone focuses on converting all isolated modules into a complete working web-based AI financial system.

By this stage:

- SMS → Structured CSV
- Spending Analysis

- Gemini AI Integration

Now we built:

A complete Web Application (HTML + CSS + JS + FastAPI + Gemini)

This milestone ensures real-time interaction between:

Frontend (User) → FastAPI Backend → AI Engine → Response → UI Display

- **Activity 4.1: FastAPI Backend Initialization**

  We created the main backend service using FastAPI.

  File:

  backend/main.py

  Initialization:

  from fastapi import FastAPI

  app = FastAPI()

  This created the main server application.

  Why FastAPI?

  - High performance
  - Easy JSON handling
  - Automatic validation
  - Clean architecture
  - Async support

  It acts as the communication bridge between frontend and AI logic.

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

from backend.genai.explanation_engine import (
    generate_general_explanation,
    explain_text,
    answer_question_with_data
)

app = FastAPI()
    You, 21 hours ago • initial commit …
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
(.venv) PS F:\SmartBridge Project\GenAI Finance Assistance> python -m uvicorn backend.main:app --reload
INFO:     Application shutdown complete.
INFO:     Finished server process [22252]
INFO:     Stopping reloader process [20804]
(.venv) PS F:\SmartBridge Project\GenAI Finance Assistance> python -m uvicorn backend.main:app --reload
INFO:     Will watch for changes in these directories: ['F:\\SmartBridge Project\\GenAI Finance Assistan
ce']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [21944] using StatReload
✅ GEMINI_API_KEY Loaded Successfully
```

- **Activity 4.2: CORS Configuration**

  Since frontend runs separately (HTML page), we enabled CORS:

  from fastapi.middleware.cors import CORSMiddleware
  app.add_middleware(
  CORSMiddleware,
  allow_origins=[""],*
  allow_credentials=True,
  allow_methods=[""],*
  allow_headers=[""],*
)

  Why?

  Because browser blocks requests from different origins without CORS.

  Without this:
  Frontend cannot call backend.

  With this:
  Frontend can communicate freely.

```python
app = FastAPI()
      You, 21 hours ago • initia
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- **Activity 4.3: API Endpoint Design**

  We created three endpoints:

  1. General Advice Endpoint

  @app.get("/explain/general")

  Flow:

  Frontend button →
  GET request →
  generate_general_explanation() →
  Gemini →
  Return JSON response.

  2. Explain Financial Text Endpoint

  @app.post("/explain/text")

  Receives:

  ```
  {
   "text": "Some financial paragraph"
  }
  ```

  Processes:

  - Injects into prompt
  - Sends to Gemini
  - Returns explanation

  3. Ask Question Endpoint

  @app.post("/ask")

  Receives:

```
{
  "question": "Where am I spending most?"
}
```

Flow:

- Load transactions.csv
- Run spending analysis
- Create summary
- Send to Gemini
- Return personalized answer

```python
@app.get("/explain/general")
def explain_general():
    explanation = generate_general_explanation()
    print("Sending to frontend:", explanation)
    return {"response": explanation}
```

```python
@app.post("/explain/text")
def explain_text_endpoint(request: TextRequest):
    explanation = explain_text(request.text)
    print("Sending to frontend:", explanation)
    return {"response": explanation}
```

```python
@app.post("/ask")
def ask_question_endpoint(request: QuestionRequest):
    answer = answer_question_with_data(request.question)
    return {"response": answer}
```

- **Activity 4.4: Frontend Development (HTML + CSS + JS)**

  Unlike previous projects (Streamlit), this project uses:

  - Pure HTML
  - CSS styling
  - JavaScript Fetch API

Frontend Structure

File:

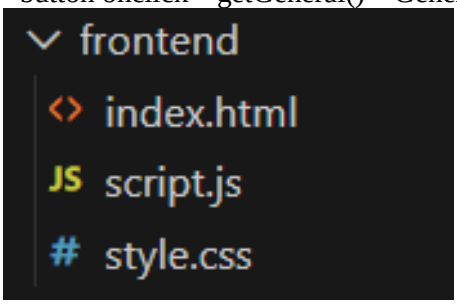
frontend/index.html

It contains:

- General Advice section
- Explain Text section
- Ask Question section

Example:

<button onclick="getGeneral()">Generate Advice</button>




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>GenAI Finance Assistant</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>💰 GenAI Finance Assistant</h1>

        <!-- General Explanation -->
        <div class="card">
            <h2>📘 General Financial Advice</h2>
            <button onclick="getGeneral()">Generate Advice</button>
            <div id="generalResult" class="result"></div>
        </div>

        <!-- Explain Financial Text -->
        <div class="card">
            <h2>📄 Explain Financial Text</h2>
            <textarea id="textInput" placeholder="Paste financial text here..."></textarea>
            <button onclick="explainText()">Explain</button>
```

```html
<html lang="en">
<body>
    <div class="container">
        <!-- Explain Financial Text -->
        <div class="card">
            <h2>📄 Explain Financial Text</h2>
            <textarea id="textInput" placeholder="Paste financial text here..."></textarea>
            <button onclick="explainText()">Explain</button>
            <div id="textResult" class="result"></div>
        </div>

        <!-- Ask Question -->
        <div class="card">
            <h2>❓ Ask a Financial Question</h2>
            <input type="text" id="questionInput" placeholder="Ask your finance question...">
            <button onclick="askQuestion()">Ask</button>
            <div id="questionResult" class="result"></div>
        </div>

    </div>

    <script src="script.js"></script>
</body>
</html>
```

- **Activity 4.5: JavaScript API Communication**

  File:

  frontend/script.js

  We used Fetch API to call backend.

  Example:

```
const response = await fetch(`${BASE_URL}/ask`, {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({ question: question })
});
```

  This sends user input to FastAPI backend.

  Then:

```
const data = await response.json();
resultDiv.innerText = data.response;
```

  Displays AI response in browser.

```
You, 22 hours ago | 1 author (You)
const BASE_URL = "http://127.0.0.1:8000";

// ================================
// General Financial Advice
// ================================
async function getGeneral() {
    const resultDiv = document.getElementById("generalResult");
    resultDiv.innerText = "Loading...";
```

- **Activity 4.6: Full Request-Response Lifecycle**

  Let's understand the full system flow:

  Step 1: User enters question
  Step 2: JS sends POST request
  Step 3: FastAPI receives request
  Step 4: Backend calls spending_analysis
  Step 5: Summary generated
  Step 6: Gemini called
  Step 7: AI generates personalized answer
  Step 8: Backend returns JSON
  Step 9: Frontend displays response

  This is complete client-server architecture.

- **Activity 4.7: Error Handling Implementation**

  We implemented multiple safety layers:

  Backend Errors

  except Exception as e:
  return f"Gemini API Error: {str(e)}"

  Frontend Errors

  catch (error) {
  resultDiv.innerText = "Backend not reachable.";
  }

  This prevents system crash.

- **Activity 4.8: Debugging & Stability Improvements**

  During integration we fixed:

- API key mismatch
- Model version issues
- DataFrame vs CSV path bug
- CORS errors
- Loading stuck issue
- 429 quota errors
- Category miscalculation
- Currency formatting issues

This debugging phase strengthened system reliability.

MILESTONE 5: Testing, Optimization & Deployment Readiness

This milestone ensures that the GenAI Finance Assistant is stable, accurate, efficient, and ready for academic evaluation or real-world deployment.

At this stage, the entire system works:

SMS → Structured CSV → Spending Analysis → Gemini AI → Web Interface

Now we validate everything professionally.

- **Activity 5.1: Functional Testing & Validation**

  In this phase, we verified that each system module works independently and correctly.

1. SMS Parsing Testing

   We tested whether raw SMS messages like:

   Rs.250 debited from A/c XX2345 on 04-09-2025 at SWIGGY.

   Are correctly converted into structured format:

   date | type | merchant | category | amount | balance

   We validated:

   - Amount extraction
   - Debit/Credit detection
   - Merchant detection
   - Date parsing
   - Balance parsing

```
Rs.45000 credited to your A/c XX2345 on 01-09-2025. Salary credited
Rs.12000 debited from A/c XX2345 on 03-09-2025 at RENT. Avl bal Rs.45000
Rs.250 debited from A/c XX2345 on 04-09-2025 at SWIGGY. Avl bal Rs.44750
Rs.120 debited from A/c XX2345 on 04-09-2025 at UBER. Avl bal Rs.44630
Rs.250 debited from A/c XX2345 on 05-09-2025 at SWIGGY. Avl bal Rs.44380
Rs.120 debited from A/c XX2345 on 05-09-2025 at UBER. Avl bal Rs.44260
Rs.250 debited from A/c XX2345 on 06-09-2025 at SWIGGY. Avl bal Rs.44010
Rs.1800 debited from A/c XX2345 on 06-09-2025 at BIGBASKET. Avl bal Rs.42210
Rs.250 debited from A/c XX2345 on 07-09-2025 at SWIGGY. Avl bal Rs.41960
Rs.2200 debited from A/c XX2345 on 07-09-2025 at AMAZON. Avl bal Rs.39760
Rs.250 debited from A/c XX2345 on 08-09-2025 at SWIGGY. Avl bal Rs.39510
Rs.120 debited from A/c XX2345 on 08-09-2025 at UBER. Avl bal Rs.39390
```

```
08-09-2025,Debited,Swiggy,Food,250,39510
08-09-2025,Debited,Uber,Transport,120,39390
09-09-2025,Debited,Swiggy,Food,250,39140
09-09-2025,Debited,Uber,Transport,120,39020
10-09-2025,Debited,Swiggy,Food,250,38770
10-09-2025,Debited,Uber,Transport,120,38650
```

2. Transaction CSV Validation

We manually verified:

- Salary credited correctly (45000)
- Rent deducted correctly (12000)
- SWIGGY categorized as Food
- UBER categorized as Transport
- AMAZON categorized as Shopping
- NETFLIX & SPOTIFY categorized as Entertainment

We also confirmed:

- No duplicate entries
- No incorrect parsing
- Proper monthly balance continuity

3. Spending Analysis Validation

We validated:

- Total Income calculation
- Total Expense calculation
- Category-wise expense calculation
- Savings calculation
- Transaction count accuracy

We cross-checked numbers manually using dataset.

This fixed earlier issue of:

 51000 food expense mismatch
✔ Correct aggregation after logic correction

```python
def analyze_spending(csv_file_path: str, month: str = None) -> dict:
    total_income = 0
    total_expense = 0
    category_expense = defaultdict(int)
    transaction_count = 0

    with open(csv_file_path, "r") as csvfile:
        reader = csv.DictReader(csvfile)

        for row in reader:
            date = row["date"]  # format: 01-09-2025

            if month:
                if f"-{month}-" not in date:
                    continue

            amount = int(row["amount"])
            tx_type = row["type"]
            category = row["category"]

            transaction_count += 1

            if tx_type == "Credited":
                total_income += amount
            elif tx_type == "Debited":
                total_expense += amount
                category_expense[category] += amount

    savings = total_income - total_expense

    return {
        "total_income": total_income,
        "total_expense": total_expense,
        "savings": savings,
        "category_expense": dict(category_expense),
        "transaction_count": transaction_count
    }
```

```
INFO:     Application startup complete.
INFO:     127.0.0.1:55274 - "OPTIONS /explain/text HTTP/1.1" 200 OK
Sending request to Gemini...
Sending to frontend: This message is simply confirming what happened with your recent investment:

1.  **"Your SIP of Rs. 5,000 in ABC Mutual Fund has been successfully processed."**
    *   **SIP:** This stands for "Systematic Investment Plan." It means you've set up a regular, fixed i
nvestment (like Rs. 5,000) that automatically gets invested into a mutual fund, usually monthly.
    *   **ABC Mutual Fund:** This is the specific investment fund you're putting your money into. A mutu
al fund is a professionally managed pool of money from many investors, which is then invested in various
 stocks, bonds, or other assets.
    *   **Successfully processed:** Your Rs. 5,000 payment went through and was invested as planned.

2.  **"Current NAV is Rs. 45.78."**
    *   **NAV:** This stands for "Net Asset Value." It's essentially the price of one single "unit" or "
share" of the mutual fund on that particular day. Think of it like the price tag for one piece of the fu
nd. This price changes daily based on the performance of the underlying investments.
```

- **Activity 5.2: AI Response Testing**

We tested Gemini integration with different scenarios:

Test Case 1: General Advice

Input:

Generate Advice

Expected:
Simple beginner financial tips

Result:
Proper structured AI response

Test Case 2: Explain Financial Text

Input:

EMI is deducted every month...

Expected:
Simple explanation

Result:
Clear explanation

Test Case 3: Data-Aware Question

Input:

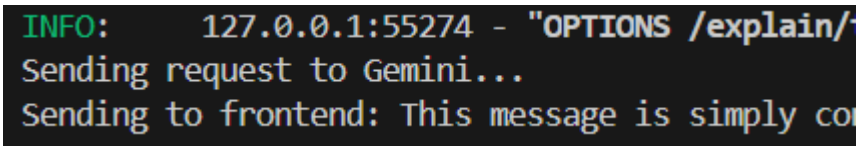
Where am I spending most?

Expected:
AI uses real transaction data

Result:
Correct category analysis
Personalized suggestions

We verified:

- AI uses real CSV summary
- No hallucinated numbers
- Currency remains in rupees
- Real data included in answer



- **Activity 5.3: Frontend Testing**

  We tested all UI sections:

  General Advice Button

  - Loading message
  - Correct response display
  - No console errors

  Explain Text Section

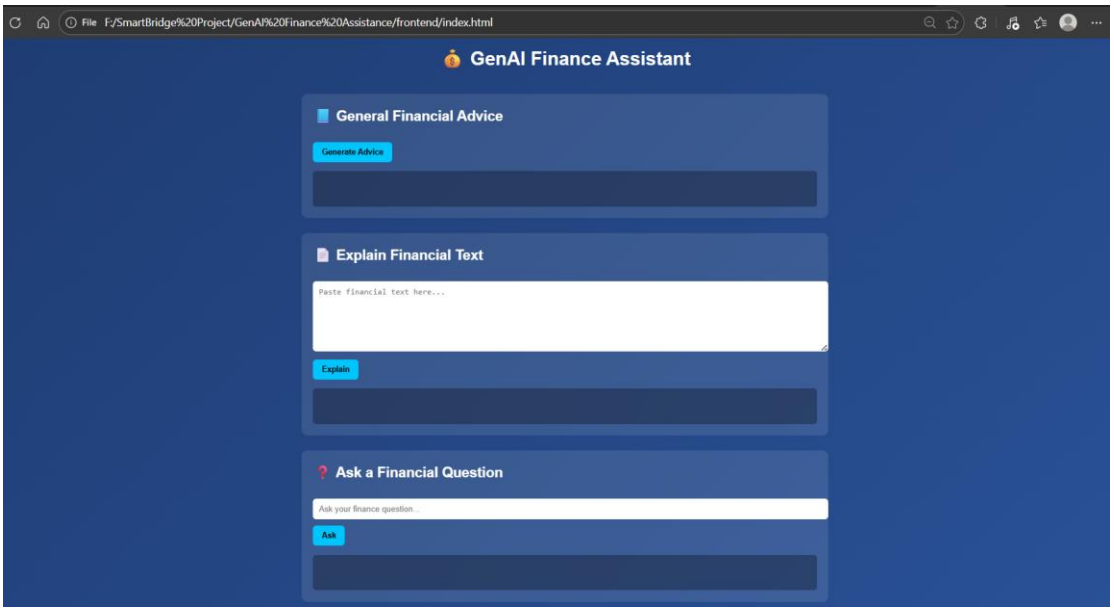
  - Empty input validation
  - Loading animation
  - Response display

  Ask Question Section

  - POST request validation
  - Backend integration working
  - Error handling functional

  We also tested:

  - Backend not running → shows "Backend not reachable"
  - Invalid input → shows proper message
  - CORS issues → resolved



- **Activity 5.4: Performance Optimization**

  We improved system efficiency by:

  1. Using lightweight Gemini model:

  gemini-2.5-flash

  2. Avoiding repeated API key initialization
  3. Handling AI errors safely
  4. Minimizing unnecessary file reads
  5. Using structured prompt formatting

  Result:

  - Faster responses
  - Lower token usage
  - Reduced quota exhaustion

- **Activity 5.5: Error Handling & Stability Improvements**

  We handled multiple critical issues during development:

Problem	Solution
API key mismatch	Environment validation
Model not found	Listed models dynamically
429 quota error	Model optimization
DataFrame path error	Corrected CSV usage

Currency confusion	Fixed prompt clarity
Category mismatch	Adjusted categorization logic
pandas module error	Installed dependency

Now system handles:

- API failure
- Missing CSV
- Invalid request
- Empty input
- Network failure

- **Activity 5.6: Deployment Readiness**

For deployment, we ensured:

- .env file security
- No API key hardcoded
- Clean folder structure
- requirements.txt prepared
- Modular architecture
- Clean separation of frontend & backend

Production Run Commands

Backend:

uvicorn backend.main:app

Frontend:
Open index.html in browser

- **Activity 5.7: Version Control & Documentation**

We structured project for GitHub:

```
GenAI Finance Assistant/
├── backend/
├── frontend/
├── data/
├── .env
├── requirements.txt
└── README.md
```

We ensured:

- Clean commit history
- Modular code
- Proper naming
- Professional documentation

CONCLUSION

The GenAI Finance Assistant successfully demonstrates how Artificial Intelligence can be integrated with real-world financial data to build a practical, intelligent, and user-friendly financial advisory system. This project combines SMS transaction parsing, structured data processing, spending analytics, and Google Gemini's generative AI capabilities into a unified application that delivers personalized financial insights.

Unlike a generic chatbot, this system does not provide random financial advice. Instead, it processes actual transactional data, calculates income, expenses, savings, and category-wise spending, and then uses AI reasoning to generate contextualized, data-driven financial responses. This makes the assistant more realistic, analytical, and aligned with real-life financial behavior.

Through this project, we achieved the following:

- Developed a complete end-to-end AI-powered web application using FastAPI and HTML/CSS/JavaScript.
- Designed a structured SMS parsing and transaction processing pipeline.
- Implemented spending pattern analysis with category-wise aggregation.
- Integrated Google Gemini (Generative AI) to provide intelligent, human-like financial guidance.
- Built a modular, scalable backend architecture suitable for future enhancement.
- Ensured secure API handling using environment configuration management.
- Implemented robust error handling and system validation mechanisms.

The project proves that combining structured financial analytics with generative AI can significantly enhance personal finance management. It provides users with clear insights into where their money is being spent, how much they are saving, and what financial improvements they can make all through an interactive AI interface.

Moreover, this system can be extended into a real-world financial assistant platform with advanced features such as automated monthly reports, predictive spending analysis, anomaly detection, budgeting recommendations, and dashboard visualizations.

In conclusion, the GenAI Finance Assistant represents a strong foundation for AI-driven financial intelligence systems. It demonstrates the practical application of data engineering, backend API development, AI prompt engineering, and frontend integration in a single cohesive solution. The project not only fulfills academic objectives but also showcases real-world problem-solving capability using modern AI technologies.